



State Machine Temel

Üç İşlem	
Metot	Etki
it(s)	yiğın ekler
pop()	üstteki çıkar
degistir(s)	pop+it

Soyut State	
<pre>class State: def __init__(self, mgr): self.manager = mgr def gir(self): pass def cik(self): pass def olaylari_isle(self,o): pass def guncelle(self, dt): pass def ciz(self, ekran): pass</pre>	

StateManager	
<pre>class StateManager: def __init__(self): self._y = [] @property def aktif(self): return (self._y[-1] if self._y else None) def it(self, s): self._y.append(s) s.gir() def pop(self): if self._y: self._y.pop().cik()</pre>	

Pause Snapshot

Pause İskelet	
<pre>class PauseState(State): def __init__(self, mgr, snapshot=None): super().__init__(mgr) self.snap = snapshot def gir(self): self.alty = pygame.Surface((W, H)) self.snap.ciz(self.alty) kar = pygame.Surface((W,H)) kar.set_alpha(140) kar.fill((0,0,0)) self.alty.blit(kar, (0,0))</pre>	

dt (Delta Time)

Frame-Bağımsız	
<pre>saat = pygame.time.Clock() dt = saat.tick(60) / 1000.0 # saniye/birim: HIZ = 240 # px/sn self.x += HIZ * dt</pre>	

Dosya Açma

Mod Karakterleri	
Mod	Anlam
"r"	oku (varsayılan)
"w"	yaz (sileri)
"a"	sona ekle
"x"	yeni oluştur
"rb"/"wb"	binary

Standart Açma	
<pre>with open(yol, "r", encoding="utf-8") as f: icerik = f.read() # with bloku otomatik close</pre>	

JSON Okuma/Yazma

Dosyadan Okuma	
<pre>import json with open(yol, "r", encoding="utf-8") as f: veri = json.load(f) # veri bir dict</pre>	

Diske Yazma (DOĞRU)	
<pre>with open(yol, "w", encoding="utf-8") as f: json.dump(veri, f, indent=2, ensure_ascii=False, sort_keys=True)</pre>	

Üç Anahtar Parametre	
Param	Etki
indent=2	insan dostu
ensure_ascii=False	Türkçe
sort_keys=True	deterministik

Atomik Yazma

.tmp + replace	
<pre>import os def atomik_yaz(yol, veri): tmp = yol.with_suffix(".json.tmp") with open(tmp, "w", encoding="utf-8") as f: json.dump(veri, f, indent=2, ensure_ascii=False) f.flush() os.fsync(f.fileno()) tmp.replace(yol)</pre>	

pathlib.Path

Yol İşlemleri	
<pre>from pathlib import Path p = (Path.home() / ".oyun" / "slot1.json") p.parent.mkdir(parents=True, exist_ok=True) p.exists() p.write_text("...", encoding="utf-8") icerik = p.read_text(encoding="utf-8") bak = p.with_suffix(".bak")</pre>	

Pickle (Dikkatli)

Sınıf Serileştirme	
<pre>import pickle # Yaz (binary): with open(yol, "wb") as f: pickle.dump(o, f) # Oku (binary): with open(yol, "rb") as f: o = pickle.load(f) # UYARI: pickle.load = kod # Sadece kendi dosyalarını ac.</pre>	

JSON vs Pickle	
Kriter	Tercih
İnsan okur	JSON
Güvenli	JSON
Sınıf otomatik	Pickle
Versiyonlama	JSON

Save Format

Tipik Save JSON

```
{
  "version": 1,
  "kayit_zamani": "...",
  "oyuncu_adi": "Ayse",
  "toplam_sure": 1820,
  "oyuncu": {...},
  "seviye": {...},
  "ilerleme": {...}
}
```

to_dict / from_dict

Sınıf-Sözlük Köprüsü

```
class Oyuncu:
    def to_dict(self):
        return {
            "x": self.x,
            "y": self.y,
            "can": self.can,
        }

    @classmethod
    def from_dict(cls, d):
        return cls(
            x=d["x"], y=d["y"],
            can=d.get("can", 5),
        )
```

Sürüm Migrasyonu

Migrate Şablonu

```
def migrate(veri):
    v = veri.get("version", 1)
    if v < 2:
        # v1 -> v2
        veri.setdefault(
            "max_can", 5)
        v = 2
    if v < 3:
        # v2 -> v3
        veri.setdefault(
            "ilerleme", {})
        v = 3
    veri["version"] = 3
    return veri
```

SaveManager

Slot Yönetimi

```
class SaveManager:
    def __init__(self, dizin):
        self.d = Path(dizin)
        self.d.mkdir(
            parents=True,
            exist_ok=True)

    def yol(self, slot):
        return (self.d /
            f"slot{slot}.json")

    def kaydet(self, slot, veri):
        # yedek + atomik
        ...

    def yukle(self, slot):
        # bozuksa .bak'a dus
        ...
```

Slot Özet Listesi

Tüm Slotlar

```
def slot_ozetleri(self, n=3):
    sonuc = []
    for i in range(1, n+1):
        yol = self.yol(i)
        if not yol.exists():
            sonuc.append(
                {"slot": i,
                 "bos": True})
            continue
        try:
            with open(yol, "r",
                encoding="utf-8") as f:
                v = json.load(f)
            sonuc.append({...})
        except (json
            .JSONDecodeError):
            sonuc.append(
                {"slot": i,
                 "bozuk": True})
    return sonuc
```

Autosave

Aralık Kontrolü

```
class SaveManager:
    AUTOSAVE_SLOT = 0
    aralik = 60.0 # sn

    def autosave_kontrol(self,
        zaman, veri):
        gecen = (zaman -
            self._son_auto)
        if gecen >= self.aralik:
            self.kaydet(
                self.AUTOSAVE_SLOT,
                **veri)
            self._son_auto = zaman
```

Settings Sınıfı

Üç Erişim

```
DEFAULTS = {
    "ses_muzik": 0.7,
    "tam_ekran": False,
    ...
}

class Settings:
    def __getitem__(self, k):
        return self._veri[k]
    def __setitem__(self, k, v):
        self._veri[k] = v
        self.kaydet()
    def __getattr__(self, k):
        if k in self._veri:
            return self._veri[k]
        raise AttributeError(k)
```

Settings Birleştir

merge+ validate

```
def yukle(self):
    if not self.yol.exists():
        return
    try:
        with open(self.yol, "r",
            encoding="utf-8") as f:
            kullanici = json.load(f)
    except (json
        .JSONDecodeError):
        return
    for k, v in kullanici.items():
        if k not in DEFAULTS:
            continue # yutulur
        if not validate(k, v):
            continue # ret
        self._veri[k] = v
```

Tuş Atama

```
Tuş İsmi

isim = pygame.key.name(
    pygame.K_SPACE)
# "space"
isim = pygame.key.name(
    settings["tus_zipla"])
ekran.blit(font.render(
    isim.upper(), True, renk),
    (x, y))

Çakışma Kontrolü

# tus_atama_modu = True iken:
def _tus_atandi(self, tus):
    for s in tus_satirlari:
        if (s.ad != mevcut and
            settings[s.ad] == tus):
            self.uyari(
                "Tus zaten atanmis")
            return
    settings[mevcut] = tus
```

Settings -> pygame

```
Anında Uygulama

def pygame_uygula(self):
    if self.sessiz:
        pygame.mixer.music \
            .set_volume(0)
    else:
        pygame.mixer.music \
            .set_volume(
                self.ses_muzik *
                self.ses_genel)
    if self.tam_ekran:
        pygame.display.set_mode(
            self.cozunurluk,
            pygame.FULLSCREEN)
```

Hata Önleme

Sık Hatalar	Çözüm
Hata	Çözüm
ensure_ascii unutuldu	False yap
KeyError eski slot	d.get(...)
Pickle saldırı	JSON kullan
encoding eksik	utf-8 ekle
Atomik değil	.tmp + replace
__setattr__ döngü	__setitem__
Autosave aşırı	30-120 sn
Migrate yok	version + setdefault